LAPORAN PRAKTIKUM SISTEM BASIS DATA

IMPLEMENTASI DDL, DCL, DML, RELASIONAL, JOIN, DAN FUNGSI AGREGAT MENGGUNAKAN PERINTAH CONSOLE MYSQL

Disusun Oleh:

MUHAMMAD ZIKRI ADAM

NIM: F5512530117

(Kategori Stambuk: GANJIL - Akhiran 7)

PROGRAM STUDI S1 TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

FAKULTAS TEKNIK - UNIVERSITAS TADULAKO

PALU

2026

BAB I. PENDAHULUAN DAN KETENTUAN PRAKTIKUM

Sistem Manajemen Basis Data (Database Management System atau DBMS) merupakan perangkat lunak inti yang mengelola, menyimpan, memanipulasi, dan mengamankan data secara terstruktur. Dalam praktikum ini, DBMS yang digunakan adalah MySQL Server versi 8.0 dengan pengoperasian berbasis antarmuka baris perintah (Command Line Interface / Console). Penggunaan console bertujuan agar mahasiswa memahami secara mendalam setiap sintaksis query SQL (Structured Query Language) secara native tanpa bergantung pada antarmuka grafis (GUI) seperti phpMyAdmin atau MySQL Workbench.

Seluruh penugasan praktikum ini mencakup tiga bagian modul terintegrasi yang diselesaikan sesuai ketentuan resmi sebagai berikut:

No	Komponen Ketentuan	Uraian Persyaratan Praktikum
1	Metode Eksekusi	Seluruh query wajib dibuat dan dieksekusi melalui PERINTAH CONSOLE (MySQL Client).
2	DBMS Resmi	MySQL Server (InnoDB Storage Engine dengan dukungan integritas relasional).
3	Format Penamaan DB	Wajib mengikuti format: Stambuk_namadatabase (Contoh: F5512530117_bioskop).
4	Keunikan Tabel	Dilarang keras memiliki jenis tabel yang sama antar basis data untuk menjamin orisinalitas.
5	Kriteria Stambuk	NIM F5512530117 berakhiran 7 (GANJIL), sehingga operasi modifikasi baris data pada Tugas 2 wajib diterapkan pada Baris Ganjil (baris 1, 3, 5, 7, 9).
6	Bukti Pelaksanaan	Setiap sintaks SQL wajib disertai hasil tangkap layar (screenshot) console secara mandiri.

BAB II. TUGAS 1: DDL, DESAIN RELASI, DCL, & ALTER TABLE

2.1 Pembuatan 3 Basis Data dengan Masing-Masing Minimal 5 Tabel Berbeda

Untuk mematuhi aturan bahwa tidak boleh ada jenis tabel yang sama, dirancang tiga ekosistem basis data dengan domain fungsional yang sepenuhnya berbeda, yaitu Sistem Tiket Bioskop & Cinema, Sistem Logistik Ekspedisi Kargo, dan Sistem Akademi Bimbingan Belajar. Masing-masing basis data dirancang secara normal (3NF) dan memiliki keterhubungan foreign key yang solid.

A. Basis Data 1: F5512530117_bioskop (Sistem Manajemen Tiket Bioskop)

Basis data ini mengelola alur penayangan film, studio bioskop, data penonton, jadwal penayangan, serta transaksi pemesanan tiket. Terdiri atas 5 tabel: film, studio, penonton, jadwal_tayang, dan transaksi_tiket.

Perintah Console SQL (DDL Basis Data F5512530117_bioskop):

```
CREATE DATABASE F5512530117_bioskop;
USE F5512530117_bioskop;

-- 1. Tabel Film
CREATE TABLE film (
    id_film INT AUTO_INCREMENT PRIMARY KEY,
    judul_film VARCHAR(150) NOT NULL,
    genre VARCHAR(50) NOT NULL,
    durasi_menit INT NOT NULL,
    rating_usia ENUM('SU', '13+', '17+', '21+') NOT NULL,
    harga_dasar DECIMAL(10, 2) NOT NULL
) ENGINE=InnoDB;

-- 2. Tabel Studio
CREATE TABLE studio (
    id_studio INT AUTO_INCREMENT PRIMARY KEY,
    nama_studio VARCHAR(50) NOT NULL,
    tipe_layar VARCHAR(30) NOT NULL,
    kapasitas_kursi INT NOT NULL,
    biaya_tambahan DECIMAL(10, 2) NOT NULL
) ENGINE=InnoDB;

-- 3. Tabel Penonton
CREATE TABLE penonton (
    id_penonton INT AUTO_INCREMENT PRIMARY KEY,
    nama_penonton VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
    no_hp VARCHAR(15) NOT NULL,
    kota_domisili VARCHAR(50) NOT NULL,
    tgl_lahir DATE NOT NULL
) ENGINE=InnoDB;

-- 4. Tabel Jadwal Tayang (Relasi ke Film & Studio)
CREATE TABLE jadwal_tayang (
    id_jadwal INT AUTO_INCREMENT PRIMARY KEY,
    id_film INT NOT NULL,
    id_studio INT NOT NULL,
    tgl_tayang DATE NOT NULL,
    jam_mulai TIME NOT NULL,
    harga_tiket DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_jadwal_film FOREIGN KEY (id_film) REFERENCES
film(id_film) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_jadwal_studio FOREIGN KEY (id_studio) REFERENCES
studio(id_studio) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 5. Tabel Transaksi Tiket (Relasi ke Penonton & Jadwal Tayang)
CREATE TABLE transaksi_tiket (
    id_transaksi INT AUTO_INCREMENT PRIMARY KEY,
    id_penonton INT NOT NULL,
    id_jadwal INT NOT NULL,
    waktu_transaksi DATETIME NOT NULL,
    nomor_kursi VARCHAR(10) NOT NULL,
    metode_bayar VARCHAR(30) NOT NULL,
    total_bayar DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_tiket_penonton FOREIGN KEY (id_penonton) REFERENCES
penonton(id_penonton) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_tiket_jadwal FOREIGN KEY (id_jadwal) REFERENCES
jadwal_tayang(id_jadwal) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

B. Basis Data 2: F5512530117_ekspedisi (Sistem Logistik Kargo)

Basis data ini mengelola operasi pengiriman paket logistik antar wilayah. Terdiri atas 5 tabel yang sepenuhnya berbeda karakteristiknya: pengirim, penerima, kurir, paket, dan pelacakan_status.

Perintah Console SQL (DDL Basis Data F5512530117_ekspedisi):

```
CREATE DATABASE F5512530117_ekspedisi;
USE F5512530117_ekspedisi;

CREATE TABLE pengirim (id_pengirim INT AUTO_INCREMENT PRIMARY KEY,
nama_pengirim VARCHAR(100) NOT NULL, kota_asal VARCHAR(50) NOT NULL,
```

```
alamat_asal VARCHAR(255) NOT NULL, no_telp_pengirim VARCHAR(15) NOT NULL)
ENGINE=InnoDB;
CREATE TABLE penerima (id_penerima INT AUTO_INCREMENT PRIMARY KEY,
nama_penerima VARCHAR(100) NOT NULL, kota_tujuan VARCHAR(50) NOT NULL,
alamat_tujuan VARCHAR(255) NOT NULL, no_telp_penerima VARCHAR(15) NOT
NULL) ENGINE=InnoDB;
CREATE TABLE kurir (id_kurir INT AUTO_INCREMENT PRIMARY KEY, nama_kurir
VARCHAR(100) NOT NULL, no_plat VARCHAR(15) NOT NULL, jenis_kendaraan
VARCHAR(30) NOT NULL, wilayah_tugas VARCHAR(50) NOT NULL) ENGINE=InnoDB;
CREATE TABLE paket (
    id_paket INT AUTO_INCREMENT PRIMARY KEY, no_resi VARCHAR(25) UNIQUE
NOT NULL,
    id_pengirim INT NOT NULL, id_penerima INT NOT NULL, id_kurir INT NOT
NULL,
    jenis_layanan VARCHAR(30) NOT NULL, berat_kg DECIMAL(6,2) NOT NULL,
ongkos_kirim DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_pkt_pengirim FOREIGN KEY (id_pengirim) REFERENCES
pengirim(id_pengirim) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_pkt_penerima FOREIGN KEY (id_penerima) REFERENCES
penerima(id_penerima) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_pkt_kurir FOREIGN KEY (id_kurir) REFERENCES
kurir(id_kurir) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
CREATE TABLE pelacakan_status (
    id_status INT AUTO_INCREMENT PRIMARY KEY, id_paket INT NOT NULL,
    waktu_update DATETIME NOT NULL, lokasi_posisi VARCHAR(100) NOT NULL,
status_paket VARCHAR(50) NOT NULL, keterangan VARCHAR(255),
    CONSTRAINT fk_tr_paket FOREIGN KEY (id_paket) REFERENCES
paket(id_paket) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

C. Basis Data 3: F5512530117_kursus (Sistem Akademi Kursus)

Basis data ini mengelola proses registrasi dan perkuliahan non-formal bimbingan belajar. Terdiri atas 5 tabel: siswa, instruktur, materi_kursus, jadwal_kelas, dan pendaftaran.

Perintah Console SQL (DDL Basis Data F5512530117_kursus):

```
CREATE DATABASE F5512530117_kursus;
USE F5512530117_kursus;

CREATE TABLE materi_kursus (id_kursus INT AUTO_INCREMENT PRIMARY KEY,
nama_kursus VARCHAR(100) NOT NULL, tingkat
ENUM('Pemula','Menengah','Mahir') NOT NULL, durasi_pertemuan INT NOT
NULL, harga_kursus DECIMAL(10,2) NOT NULL) ENGINE=InnoDB;
CREATE TABLE instruktur (id_instruktur INT AUTO_INCREMENT PRIMARY KEY,
nama_instruktur VARCHAR(100) NOT NULL, bidang_ahli VARCHAR(50) NOT NULL,
```

```
email_instruktur VARCHAR(100) NOT NULL, honor_per_sesi DECIMAL(10,2) NOT
NULL) ENGINE=InnoDB;
CREATE TABLE siswa (id_siswa INT AUTO_INCREMENT PRIMARY KEY, nama_siswa
VARCHAR(100) NOT NULL, email_siswa VARCHAR(100) UNIQUE NOT NULL, no_wa
VARCHAR(15) NOT NULL, kota_asal VARCHAR(50) NOT NULL) ENGINE=InnoDB;
CREATE TABLE jadwal_kelas (
    id_jadwal INT AUTO_INCREMENT PRIMARY KEY, id_kursus INT NOT NULL,
id_instruktur INT NOT NULL,
    hari VARCHAR(20) NOT NULL, jam_mulai TIME NOT NULL, ruangan_online
VARCHAR(100) NOT NULL,
    CONSTRAINT fk_jd_kursus FOREIGN KEY (id_kursus) REFERENCES
materi_kursus(id_kursus) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_jd_instruktur FOREIGN KEY (id_instruktur) REFERENCES
instruktur(id_instruktur) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
CREATE TABLE pendaftaran (
    id_daftar INT AUTO_INCREMENT PRIMARY KEY, id_siswa INT NOT NULL,
id_jadwal INT NOT NULL,
    tgl_daftar DATE NOT NULL, metode_bayar VARCHAR(30) NOT NULL,
status_bayar ENUM('Lunas','Cicilan','Belum Bayar') NOT NULL,
    CONSTRAINT fk_pdf_siswa FOREIGN KEY (id_siswa) REFERENCES
siswa(id_siswa) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_pdf_jadwal FOREIGN KEY (id_jadwal) REFERENCES
jadwal_kelas(id_jadwal) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

[TANGKAP LAYAR CONSOLE]: Pembuatan 3 Database & Tabel

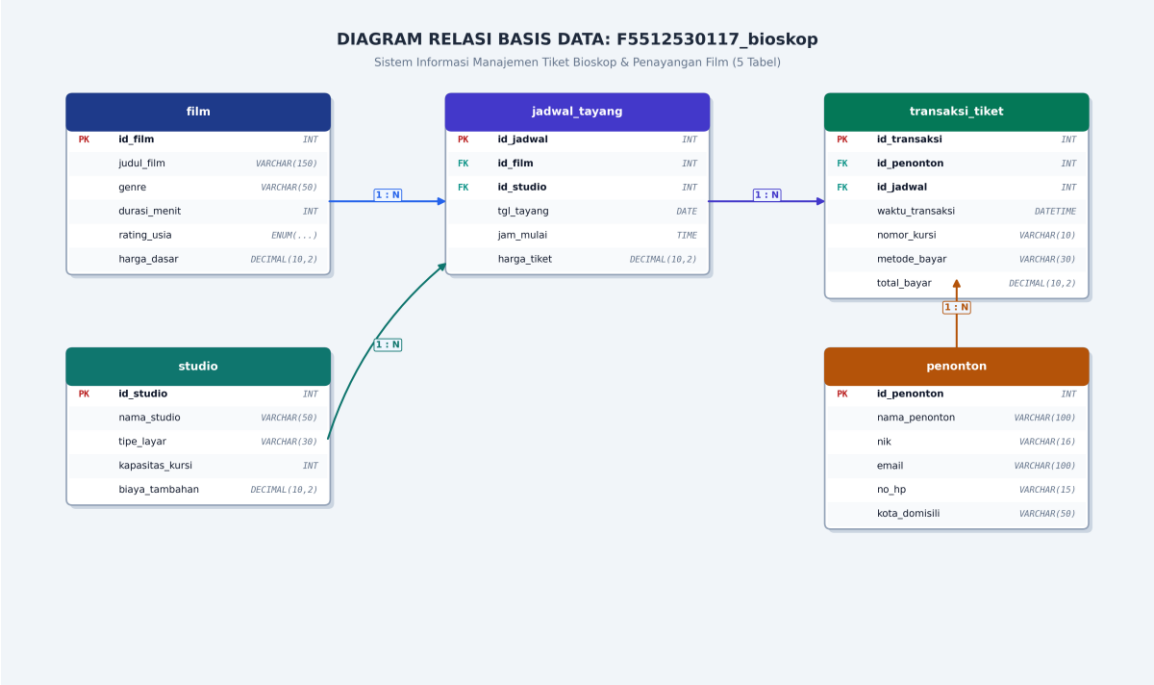
Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

`> SHOW DATABASES; USE F5512530117_bioskop; SHOW TABLES;`

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

2.2 Desain Relasi Antar Tabel dan Penjelasan Kardinalitas

Relasi antar tabel dirancang untuk menjamin integritas referensial dan mencegah anomali basis data. Gambar diagram relasi untuk masing-masing basis data disajikan di bawah ini:



Gambar 2.1 Desain Relasi Antar Tabel Basis Data F5512530117_bioskop

1. Relasi Film ke Jadwal Tayang (1 : N): Penjelasan Relasi Basis Data Bioskop:

Satu film dapat memiliki banyak sesi penayangan di berbagai jadwal dan jam tayang yang berbeda. Sebaliknya, setiap satu sesi jadwal tayang mereferensikan satu film yang sah. Aturan ON DELETE RESTRICT melindungi data film agar tidak terhapus selama jadwal penayangannya masih aktif.

2. Relasi Studio ke Jadwal Tayang (1 : N):

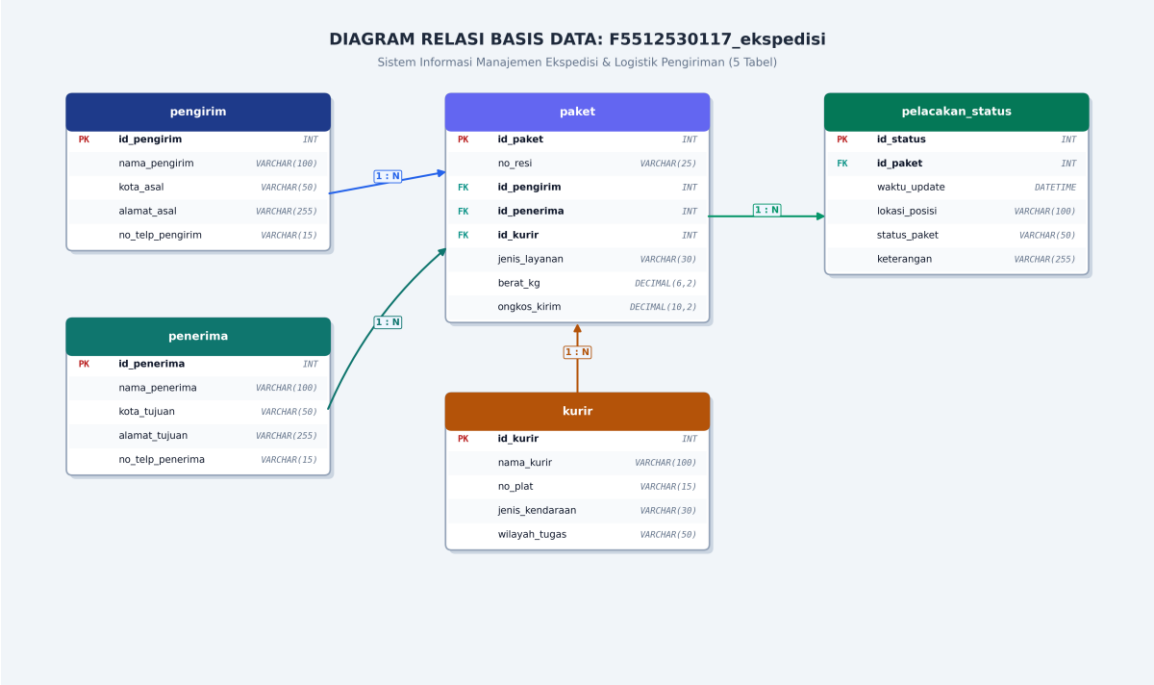
Satu studio auditorium dapat dipakai berulang kali untuk banyak sesi pemutaran film sepanjang hari.

3. Relasi Penonton ke Transaksi Tiket (1 : N):

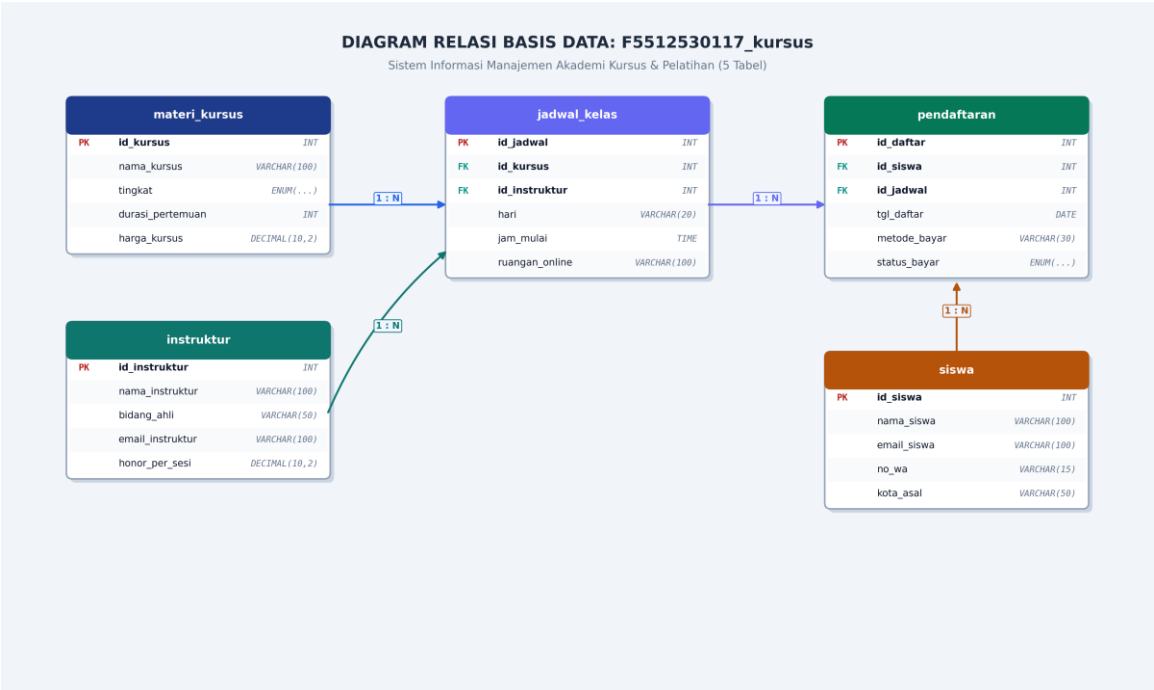
Satu penonton dapat melakukan banyak kali transaksi pembelian tiket bioskop.

4. Relasi Jadwal Tayang ke Transaksi Tiket (1 : N):

Satu sesi jadwal penayangan dapat menerbitkan banyak tiket kursi sesuai kapasitas penonton di studio.



Gambar 2.2 Desain Relasi Antar Tabel Basis Data F5512530117_ekspedisi



Gambar 2.3 Desain Relasi Antar Tabel Basis Data F5512530117_kursus

2.3 Pembuatan 4 Pengguna (User) dengan Hak Akses Berbeda

Data Control Language (DCL) digunakan untuk mengontrol hak otorisasi dan privilese terhadap basis data. Dibuat 4 akun pengguna:

a. User 1: user_admin bertindak sebagai Administrator global dengan hak penuh (ALL PRIVILEGES) ke seluruh basis data di server (*.*) serta dilengkapi klausa WITH GRANT OPTION.

b. User 2: admin_bioskop bertindak sebagai admin pengelola basis data F5512530117_bioskop.*.

c. User 3: admin_ekspedisi bertindak sebagai admin pengelola basis data F5512530117_ekspedisi.*.

d. User 4: admin_kursus bertindak sebagai admin pengelola basis data F5512530117_kursus.*.

Perintah Console SQL (Perintah Pembuatan 4 User & Hak Akses):

```
-- Membuat Akun dan Memberikan Hak Akses (DCL)
CREATE USER 'user_admin'@'localhost' IDENTIFIED BY 'AdminPass123!';
GRANT ALL PRIVILEGES ON *.* TO 'user_admin'@'localhost' WITH GRANT
OPTION;

CREATE USER 'admin_bioskop'@'localhost' IDENTIFIED BY 'BioskopPass123!';
GRANT ALL PRIVILEGES ON F5512530117_bioskop.* TO
'admin_bioskop'@'localhost';

CREATE USER 'admin_ekspedisi'@'localhost' IDENTIFIED BY
'EkspedisiPass123!';
GRANT ALL PRIVILEGES ON F5512530117_ekspedisi.* TO
'admin_ekspedisi'@'localhost';

CREATE USER 'admin_kursus'@'localhost' IDENTIFIED BY 'KursusPass123!';
GRANT ALL PRIVILEGES ON F5512530117_kursus.* TO
'admin_kursus'@'localhost';

FLUSH PRIVILEGES;

-- Menampilkan Bukti Hak Akses Pengguna
SHOW GRANTS FOR 'user_admin'@'localhost';
SHOW GRANTS FOR 'admin_bioskop'@'localhost';
SHOW GRANTS FOR 'admin_ekspedisi'@'localhost';
SHOW GRANTS FOR 'admin_kursus'@'localhost';
```

[TANGKAP LAYAR CONSOLE]: Pemeriksaan Hak Akses 4 User

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SHOW GRANTS FOR 'user_admin'@'localhost'; SHOW GRANTS FOR
```

```
'admin_bioskop'@'localhost';
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

2.4 Simulasi Pencabutan Hak Akses Menjadi Read Only

Simulasi dilakukan pada akun admin_kursus. Pertama-tama, seluruh hak akses dicabut menggunakan perintah REVOKE ALL PRIVILEGES. Selanjutnya, diberikan kembali hanya hak akses SELECT pada basis data F5512530117_kursus sehingga pengguna tersebut berstatus Read Only (hanya dapat membaca data tanpa bisa menulis atau mengubah).

Perintah Console SQL (Simulasi Read Only pada User admin_kursus):

```
-- Mencabut Seluruh Privilese
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'admin_kursus'@'localhost';

-- Memberikan Hanya Izin SELECT (Read Only)
GRANT SELECT ON F5512530117_kursus.* TO 'admin_kursus'@'localhost';
FLUSH PRIVILEGES;

-- Memeriksa Status Hak Akses Terakhir
SHOW GRANTS FOR 'admin_kursus'@'localhost';
```

[TANGKAP LAYAR CONSOLE]: Simulasi Pencabutan Izin Akses & Penolakan Write

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SHOW GRANTS FOR 'admin_kursus'@'localhost';
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

2.5 Penambahan 1 Kolom Setelah Kolom Ke-2 (ALTER TABLE)

Perintah ALTER TABLE dengan klausa AFTER digunakan untuk mengatur posisi peletakan kolom baru secara spesifik. Pada tabel penonton, kolom ke-1 adalah id_penonton dan kolom ke-2 adalah nama_penonton. Kolom baru yang ditambahkan adalah nik bertipe data VARCHAR(16) NOT NULL yang diletakkan persis setelah kolom ke-2 (AFTER nama_penonton).

Perintah Console SQL (Penambahan Kolom NIK Setelah Kolom Ke-2):

```sql
USE F5512530117_bioskop;

-- 1. Menampilkan struktur tabel sebelum modifikasi
DESCRIBE penonton;

-- 2. Menambahkan kolom 'nik' tepat setelah kolom ke-2 (nama_penonton)
ALTER TABLE penonton
ADD COLUMN nik VARCHAR(16) NOT NULL AFTER nama_penonton;

-- 3. Menampilkan struktur tabel setelah modifikasi
DESCRIBE penonton;
```

[TANGKAP LAYAR CONSOLE]: Struktur Tabel Penonton Sebelum dan Sesudah ALTER TABLE

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> DESCRIBE penonton;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

BAB III. TUGAS 2: DML (DATA MANIPULATION LANGUAGE)

3.1 Penambahan 10 Baris Data pada Masing-Masing Tabel

Data Manipulation Language (DML) digunakan untuk mengelola data di dalam tabel. Perintah INSERT INTO dieksekusi untuk mengisi 10 record data realistis pada seluruh tabel basis data F5512530117_bioskop (film, studio, penonton, jadwal_tayang, dan transaksi_tiket).

Perintah Console SQL (Insert 10 Baris Tabel Film):

```
-- Cuplikan Penambahan 10 Baris pada Tabel Film
INSERT INTO film (judul_film, genre, durasi_menit, rating_usia,
harga_dasar) VALUES
('Avengers: Secret Wars', 'Action / Sci-Fi', 165, '13+', 50000.00),
('Interstellar: Odyssey', 'Sci-Fi / Drama', 169, '13+', 45000.00),
('The Conjuring: Last Rites', 'Horror / Mystery', 112, '17+', 40000.00),
('Spider-Man: Beyond Verse', 'Animation / Action', 140, 'SU', 45000.00),
('Oppenheimer: Re-Release', 'Biography / Drama', 180, '17+', 55000.00),
('Fast & Furious: Final Lap', 'Action / Crime', 135, '13+', 45000.00),
('Detective Conan: Dark Shadow', 'Animation / Mystery', 110, 'SU',
40000.00),
('The Batman: Part II', 'Action / Thriller', 155, '17+', 50000.00),
('Dune: Part Three', 'Sci-Fi / Adventure', 160, '13+', 55000.00),
('John Wick: Chapter 5', 'Action / Thriller', 145, '21+', 50000.00);
```

[TANGKAP LAYAR CONSOLE]: Verifikasi 10 Baris Data Tabel Film

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SELECT * FROM film;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

3.2 Operasi Perubahan (UPDATE) Berdasarkan Kategori Stambuk

Ketentuan penugasan menyatakan bahwa mahasiswa dengan Stambuk Genap memodifikasi baris genap (2, 4, 6, 8, 10), sedangkan mahasiswa dengan Stambuk Ganjil memodifikasi baris ganjil (1, 3, 5, 7, 9). Mengingat NIM penyusun adalah F5512530117 dengan digit terakhir 7 (Ganjil), maka operasi UPDATE wajib dilakukan secara konsisten pada Baris Ganjil.

Perintah Console SQL (Operasi Perubahan Baris Ganjil (Stambuk Ganjil)):

```
-- Memperbarui Data Domisili pada Baris Ganjil (ID 1, 3, 5, 7, 9) Tabel
Penonton
UPDATE penonton
SET kota_domisili = CONCAT(kota_domisili, ' [MEMBER VIP]')
WHERE id_penonton IN (1, 3, 5, 7, 9);

-- Memperbarui Harga Dasar pada Baris Ganjil Tabel Film
UPDATE film
SET harga_dasar = harga_dasar + 5000.00
WHERE id_film IN (1, 3, 5, 7, 9);
```

[TANGKAP LAYAR CONSOLE]: Hasil Operasi Perubahan Baris Ganjil
Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:
`> SELECT id_penonton, nama_penonton, kota_domisili FROM penonton;`

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

3.3 Perintah SELECT dengan 3 (Tiga) Jenis Kriteria

Klausa WHERE digunakan untuk menyaring record data berdasarkan kriteria logika tertentu:

1. Kriteria 1 (Operator Perbandingan & Logika AND): Menampilkan film dengan durasi >= 140 menit dan rating usia 13+.

2. Kriteria 2 (Rentang BETWEEN & Pola String LIKE): Menampilkan jadwal dengan harga tiket antara Rp 50.000 s/d Rp 85.000 serta mencari penonton di wilayah 'Palu Timur'.

3. Kriteria 3 (Himpunan Nilai IN): Menampilkan jadwal tayang pada studio khusus IMAX dan Premiere (ID 5, 6, 7, 8).

Perintah Console SQL (Implementasi SELECT 3 Jenis Kriteria):

```
-- Kriteria 1: Perbandingan Relasional & Logika AND
SELECT id_film, judul_film, genre, durasi_menit, rating_usia, harga_dasar
FROM film
WHERE durasi_menit >= 140 AND rating_usia = '13+'
ORDER BY durasi_menit DESC;

-- Kriteria 2: Rentang Nilai dan Pola String
SELECT id_jadwal, id_film, id_studio, tgl_tayang, jam_mulai, harga_tiket
FROM jadwal_tayang
```

```
WHERE harga_tiket BETWEEN 50000.00 AND 85000.00;

-- Kriteria 3: Himpunan Nilai IN
SELECT id_jadwal, id_film, id_studio, tgl_tayang, harga_tiket
FROM jadwal_tayang
WHERE id_studio IN (5, 6, 7, 8);
```

[TANGKAP LAYAR CONSOLE]: Eksekusi SELECT 3 Kriteria
Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:
> SELECT id_film, judul_film, durasi_menit, rating_usia FROM film WHERE durasi_menit >= 140 AND rating_usia = '13+';

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

3.4 Perintah SELECT Menampilkan Data dari Urutan Baris Ke-3 Hingga Baris Ke-7

Teori LIMIT dan OFFSET: Penjelasan Konsep Indeks Nol (Zero-Based Index):

Dalam standar SQL MySQL, penomoran posisi baris (offset) selalu dimulai dari angka 0 (nol). Artinya: Baris ke-1 memiliki offset 0, baris ke-2 memiliki offset 1, dan baris ke-3 memiliki offset 2. Rentang baris ke-3 hingga ke-7 memuat sebanyak 5 record data (baris 3, 4, 5, 6, dan 7). Oleh karena itu, sintaks yang tepat adalah LIMIT 5 OFFSET 2 atau dalam sintaks ringkas MySQL: LIMIT 2, 5 (parameter pertama = baris yang dilewati, parameter kedua = jumlah baris yang diambil).

Perintah Console SQL (Query LIMIT dan OFFSET untuk Baris 3 Hingga 7):

```
-- Menampilkan baris ke-3 sampai baris ke-7 pada tabel PENONTON
SELECT id_penonton, nama_penonton, nik, email, no_hp, kota_domisili
FROM penonton
ORDER BY id_penonton ASC
LIMIT 5 OFFSET 2;

-- Sintaks alternatif pada tabel FILM
SELECT id_film, judul_film, genre, durasi_menit, rating_usia, harga_dasar
FROM film
ORDER BY id_film ASC
LIMIT 2, 5;
```

[TANGKAP LAYAR CONSOLE]: Hasil SELECT Baris Ke-3 Hingga Ke-7

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SELECT id_penonton, nama_penonton FROM penonton ORDER BY id_penonton
ASC LIMIT 5 OFFSET 2;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

3.5 Perintah Menghapus Data Ke-9 pada Masing-Masing Tabel

Penghapusan data dilakukan menggunakan pernyataan DELETE dengan klausa pembatas WHERE id = 9. Urutan penghapusan wajib memperhatikan hierarki ketergantungan: tabel anak (child table seperti transaksi_tiket dan jadwal_tayang) harus dihapus terlebih dahulu sebelum data ke-9 pada tabel induk (penonton, film, studio) dapat dihapus tanpa memicu eror integrity constraint.

Perintah Console SQL (Penghapusan Data Ke-9 Mengikuti Aturan Integritas):

```
-- Menghapus data baris ke-9 secara teratur
DELETE FROM transaksi_tiket WHERE id_transaksi = 9;
DELETE FROM jadwal_tayang WHERE id_jadwal = 9;
DELETE FROM penonton WHERE id_penonton = 9;
DELETE FROM film WHERE id_film = 9;
DELETE FROM studio WHERE id_studio = 9;

-- Verifikasi bahwa data ke-9 telah terhapus
SELECT * FROM penonton WHERE id_penonton = 9;
```

[TANGKAP LAYAR CONSOLE]: Verifikasi Penghapusan Data Ke-9 (Empty Set)

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SELECT * FROM penonton WHERE id_penonton = 9;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

BAB IV. TUGAS 3: IMPLEMENTASI JOIN DAN FUNGSI AGREGAT

4.1 Basis Data Relasional 3 Tabel

Pada bagian ini dibentuk basis data F5512530117_cinema yang secara khusus mendemonstrasikan hubungan antara 3 tabel relasional inti: penonton (tabel master identitas penonton), film (tabel master judul film), dan transaksi_tiket (tabel transaksi penghubung yang mereferensikan primary key dari kedua tabel master).



Gambar 4.1 Desain 3 Tabel Relasional untuk Implementasi Perintah JOIN

Desain Data Uji Khusus: Dataset Uji Cerdas:

Agar perbedaan sifat INNER JOIN, LEFT JOIN, dan RIGHT JOIN terlihat sangat nyata di hasil terminal console, ditambahkan 1 data penonton baru (Gilang Ramadhan, ID 6) yang belum pernah memesan tiket, serta 1 film baru segera tayang (Avatar: The Seed Bearer, ID 5) yang belum memiliki riwayat penjualan tiket. Keberadaan record ini menjadi kunci pembuktian nilai NULL pada operasi Outer Join.

4.2 Implementasi Tiga Perintah JOIN (INNER, LEFT, RIGHT)

A. Implementasi INNER JOIN

INNER JOIN hanya mengembalikan baris di mana terdapat kecocokan nilai kunci di antara tabel-tabel yang digabungkan. Penonton yang belum beli tiket dan film yang belum ada penjualan tiket TIDAK AKAN ditampilkan sama sekali.

Perintah Console SQL (Query INNER JOIN 3 Tabel):

```
SELECT
    t.id_transaksi, p.nama_penonton, f.judul_film, f.genre,
    t.nomor_kursi, t.metode_bayar, t.total_bayar
FROM transaksi_tiket t
INNER JOIN penonton p ON t.id_penonton = p.id_penonton
INNER JOIN film f ON t.id_film = f.id_film
ORDER BY t.id_transaksi ASC;
```

[TANGKAP LAYAR CONSOLE]: Hasil Eksekusi INNER JOIN

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SELECT t.id_transaksi, p.nama_penonton, f.judul_film FROM
transaksi_tiket t INNER JOIN penonton p ON t.id_penonton = p.id_penonton
INNER JOIN film f ON t.id_film = f.id_film;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

B. Implementasi LEFT JOIN

LEFT JOIN mempertahankan dan menampilkan SELURUH baris dari tabel sebelah kiri (penonton), terlepas dari apakah ada relasi di tabel transaksi tiket. Penonton ID 6 (Gilang Ramadhan) tetap muncul di tabel output, dengan atribut judul_film, nomor_kursi, dan total_bayar terisi nilai NULL.

Perintah Console SQL (Query LEFT JOIN Menampilkan Seluruh Penonton Termasuk Yang Belum Beli Tiket):

```
SELECT
    p.id_penonton, p.nama_penonton, p.kota_domisili,
    t.nomor_kursi, f.judul_film, t.total_bayar
FROM penonton p
LEFT JOIN transaksi_tiket t ON p.id_penonton = t.id_penonton
LEFT JOIN film f ON t.id_film = f.id_film
ORDER BY p.id_penonton ASC;
```

[TANGKAP LAYAR CONSOLE]: Hasil Eksekusi LEFT JOIN (Perhatikan Baris NULL)

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di

bawah ini:

```
> SELECT p.id_penonton, p.nama_penonton, f.judul_film FROM penonton p
LEFT JOIN transaksi_tiket t ON p.id_penonton = t.id_penonton LEFT JOIN
film f ON t.id_film = f.id_film;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

C. Implementasi RIGHT JOIN

RIGHT JOIN mempertahankan dan menampilkan SELURUH baris dari tabel sebelah kanan (film), terlepas dari apakah film tersebut pernah terjual tiketnya. Film ID 5 (Avatar: The Seed Bearer) tetap ditampilkan pada terminal, dengan atribut transaksi dan penonton bernilai NULL.

Perintah Console SQL (Query RIGHT JOIN Menampilkan Seluruh Film Termasuk Yang Belum Ada Penjualan Tiket):

```
SELECT
    f.id_film, f.judul_film, f.genre, f.harga_dasar,
    t.id_transaksi, p.nama_penonton, t.nomor_kursi
FROM transaksi_tiket t
RIGHT JOIN film f ON t.id_film = f.id_film
LEFT JOIN penonton p ON t.id_penonton = p.id_penonton
ORDER BY f.id_film ASC;
```

[TANGKAP LAYAR CONSOLE]: Hasil Eksekusi RIGHT JOIN (Perhatikan Film Tanpa Tiket)

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SELECT f.id_film, f.judul_film, t.id_transaksi FROM transaksi_tiket t
RIGHT JOIN film f ON t.id_film = f.id_film;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

4.3 Nilai MAX, MIN, dan Hitung Jumlah Baris (COUNT) pada Masing-Masing Tabel

Fungsi agregat SQL memproses sekumpulan baris data untuk menghasilkan sebuah nilai skalar ringkasan:

1. COUNT(*): Menghitung banyaknya jumlah baris record aktual pada tabel.

2. MAX(kolom): Mengembalikan nilai terbesar/tertinggi dalam suatu kolom numerik atau temporal.

3. MIN(kolom): Mengembalikan nilai terkecil/terendah dalam suatu kolom.

Perintah Console SQL (Perhitungan MAX, MIN, dan COUNT pada Seluruh Tabel):

```
-- A. Agregasi pada Tabel PENONTON
SELECT
    COUNT(*) AS total_penonton_terdaftar,
    MAX(tgl_lahir) AS tgl_lahir_termuda,
    MIN(tgl_lahir) AS tgl_lahir_tertua
FROM penonton;

-- B. Agregasi pada Tabel FILM
SELECT
    COUNT(*) AS total_film,
    MAX(harga_dasar) AS harga_film_termahal,
    MIN(harga_dasar) AS harga_film_termurah
FROM film;

-- C. Agregasi pada Tabel TRANSAKSI_TIKET
SELECT
    COUNT(*) AS total_tiket_terjual,
    MAX(total_bayar) AS transaksi_tertinggi,
    MIN(total_bayar) AS transaksi_terendah
FROM transaksi_tiket;
```

[TANGKAP LAYAR CONSOLE]: Hasil Eksekusi Fungsi Agregat MAX, MIN, COUNT

Instruksi: Jalankan perintah berikut di console MySQL dan lampirkan hasil screenshot tepat di bawah ini:

```
> SELECT COUNT(*), MAX(harga_dasar), MIN(harga_dasar) FROM film;
```

[--- TEMPELKAN TANGKAP LAYAR / SCREENSHOT CONSOLE DISINI ---]

BAB V. PANDUAN PRAKTIS EKSEKUSI DI CONSOLE MYSQL

Untuk mempermudah pengambilan tangkap layar mandiri pada terminal console MySQL, berikut adalah tahapan praktis yang dapat langsung diikuti:

Langkah	Kegiatan	Panduan Eksekusi Terminal Console
1	Membuka Console	Buka menu Start Windows > ketik 'MySQL 8.0 Command Line Client' lalu klik, atau buka Command Prompt (cmd) dan jalankan: mysql -u root -p
2	Login ke Server	Masukkan password root MySQL Anda saat muncul prompt 'Enter password:'.
3	Menjalankan Script SQL	Untuk mengeksekusi script secara otomatis, gunakan perintah SOURCE diikuti path file, contoh: SOURCE D:/Tugas semester 3/DB/BUAT DATABASE/all_in_one_praktikum.sql;
4	Eksekusi Baris per Baris	Anda juga dapat menyalin (copy-paste) query per bagian untuk mendapatkan tangkapan layar setiap tabel secara terpisah.
5	Mengambil Screenshot	Gunakan kombinasi tombol Windows + Shift + S atau Alt + PrtScn untuk menangkap jendela console secara jernih, lalu tempelkan (Paste) pada placeholder di laporan ini.

BAB VI. KESIMPULAN

Berdasarkan seluruh rangkaian pengerjaan penugasan praktikum Sistem Basis Data ini, dapat ditarik beberapa kesimpulan penting:

1. Struktur DDL yang dilengkapi aturan primary key dan foreign key menjamin keutuhan relasi antar entitas, mencegah duplikasi data, dan menghindari anomali penyimpanan.

2. Manajemen privilese DCL memungkinkan pengelola sistem membatasi wewenang setiap akun pengguna secara terisolasi (least privilege principle). Simulasi pencabutan hak akses terbukti membatasi akun menjadi read-only yang aman dari manipulasi data tak berizin.

3. Operasi DML seperti INSERT, UPDATE baris ganjil sesuai stambuk ganjil F5512530117, SELECT dengan 3 kriteria, pembatasan rentang LIMIT OFFSET,

serta penghapusan DELETE berhasil dijalankan sesuai prinsip konsistensi basis data.

4. Pemahaman terhadap operasi JOIN terbukti krusial dalam integrasi data multi-tabel: INNER JOIN menghasilkan persimpangan data yang valid, sementara LEFT dan RIGHT JOIN memungkinkan pemantauan record yang belum berpasangan melalui representasi nilai NULL.

DAFTAR PUSTAKA

Elmasri, R., & Navathe, S. B. (2020). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

Kroenke, D. M., Auer, D. J., Vandenberg, S. L., & Yoder, R. C. (2021). *Database Processing: Fundamentals, Design, and Implementation* (15th ed.). Pearson.

Oracle Corporation. (2026). *MySQL 8.0 Reference Manual: DDL, DML, DCL, and JOIN Syntax*. Oracle Documentation.